Database Management System
Mini Project

HOTEL MANAGEMENT SYSTEM

Team Member 1:	Akilesh Gowda K S
SRN Member 1:	PES2UG23CS040
Team Member 2:	Akshay
SRN Member 2:	PES2UG23CS044

Institution:	PES University
Department:	Computer Science and Engineering
Course:	Database Management Systems
Semester:	5th Semester (2025-2026)

ABSTRACT

The Hotel Management System is a comprehensive database-driven web application designed to streamline and automate the operations of hotel businesses. This system provides an integrated platform for managing hotels, guest information, room bookings, payments, staff records, restaurant orders, guest services, and customer feedback. Built using Python's Streamlit framework for the frontend and MySQL for the backend database, the application offers an intuitive user interface with real-time data visualization and analytics capabilities.

The system implements advanced database concepts including triggers for automated room status updates, stored procedures for complex business logic, user-defined functions for calculations, and comprehensive CRUD operations. It features a modern, responsive web interface with gradient designs, interactive charts, and dynamic dashboards that provide insights into hotel performance, revenue trends, and operational metrics. The application ensures data integrity through foreign key constraints, supports multiple payment methods, and maintains detailed logs of all transactions and customer interactions.

TABLE OF CONTENTS

1. User Requirement Specification

2. Functional Requirements

3. Software, Tools, and Technologies Used

4. ER Diagram

5. Relational Schema

6. DDL Commands

7. DML Commands - INSERT Operations

8. CRUD Operation Screenshots

9. Application Features and Functionalities

10. Triggers, Procedures, Functions, and Advanced Queries

11. Code Snippets for Invoking Database Operations

12. Complete SQL Queries Reference

13. GitHub Repository

14. Conclusion

1. USER REQUIREMENT SPECIFICATION

1.1 Purpose of the Project

The Hotel Management System is designed to address the operational challenges faced by hotel businesses in managing their day-to-day activities efficiently. Traditional manual methods of handling hotel operations are time-consuming, error-prone, and lack real-time insights into business performance. This project aims to create a centralized, automated system that integrates all aspects of hotel management into a single cohesive platform.

The primary purpose is to provide hotel administrators and staff with tools to efficiently manage multiple hotels, track room availability, process guest bookings, handle payments, manage staff assignments, coordinate restaurant services, and collect customer feedback. Additionally, the system provides powerful analytics and reporting capabilities that help management make data-driven decisions to improve service quality and maximize revenue.

1.2 Scope of the Project

The Hotel Management System encompasses a wide range of functionalities covering all major aspects of hotel operations. The system manages multiple hotels within a chain, each with its own set of rooms, staff, and services. It handles the complete guest lifecycle from registration and booking to check-out and feedback collection. The payment module supports multiple payment methods (Cash, Card, UPI) and maintains a complete financial audit trail.

The scope extends to staff management, integrated restaurant ordering, comprehensive service management, and advanced analytics dashboard. The web-based interface ensures accessibility from any device with a browser, making it suitable for both on-premise and remote management.

2. FUNCTIONAL REQUIREMENTS

2.1 Hotel Management

• Add New Hotel: Register new hotels with name, location, and rating
• View Hotels: Display all hotels in organized table format
• Update Hotel Information: Modify hotel details
• Delete Hotel: Remove hotels with cascade delete
• Hotel Analytics: View performance metrics

2.2 Room Management

• Add Rooms: Create rooms with type and pricing
• View Rooms: Display room inventory with status
• Update Room Details: Modify room information
• Check Availability: Filter available rooms
• Auto-update Status: Automatic status changes via triggers

2.3 Guest Management

• Register Guest: Add new guest records
• View All Guests: Display guest directory
• Update Guest Info: Modify guest details
• Guest History: View booking and service history
• Service Assignment: Link guests with services

2.4 Booking Management

• Create Booking: Make new reservations
• View Bookings: Display all reservations
• Check-in Process: Mark guests as checked in
• Check-out Process: Release rooms to available status
• Calculate Stay Duration: Auto-compute nights and charges

2.5 Payment Management

• Record Payment: Enter payment transactions
• View Payment History: Display all payments
• Payment Validation: Ensure amounts don't exceed totals
• Multiple Payment Methods: Support Cash, Card, UPI

• Payment Logging: Automatic audit trail creation

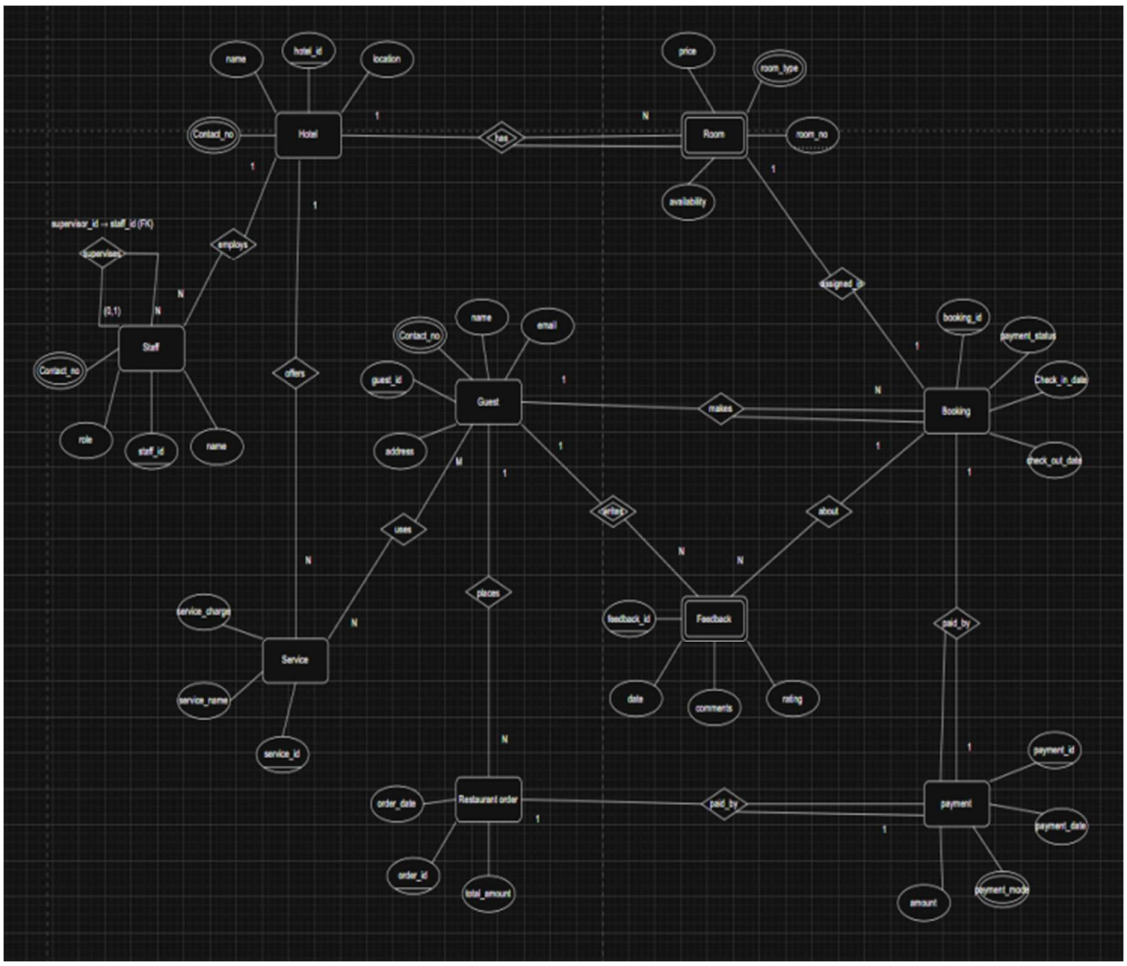
2.6 Analytics and Reporting

• Dashboard Metrics: Display key statistics
• Revenue Trends: Visualize revenue over time
• Room Occupancy Analysis: Show booking distribution
• Top Hotels Report: Identify best performers
• Service Popularity: Analyze service usage

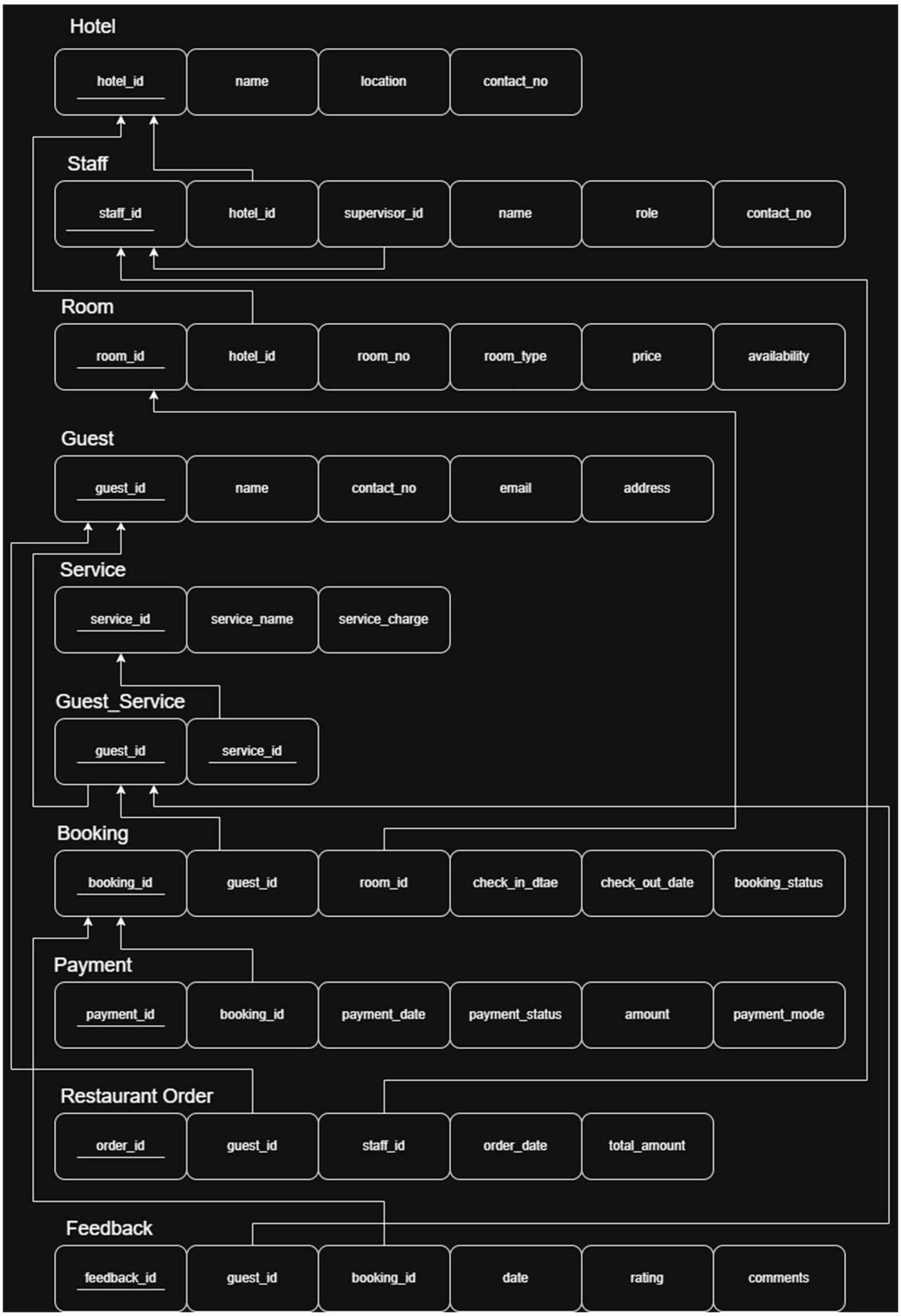
3. SOFTWARE, TOOLS, AND TECHNOLOGIES USED

Category	Technology	Version/Details
Database	MySQL	8.0+
Programming Language	Python	3.12
Web Framework	Streamlit	1.31.0
Database Connector	mysql-connector-python	8.3.0
Data Processing	Pandas	2.2.0
Visualization	Plotly	5.18.0
Development IDE	Visual Studio Code	Latest
Version Control	Git & GitHub	Latest

4. ER DIAGRAM


name
hotel_id
location
Contact_no
Hotel
1
has
N
Room
price
room_type
room_no
availability
1
1
supervisor_id → staff_id (FK)
supervises
employs
(0,1)
N
N
Staff
Contact_no
role
staff_id
name
offers
assigned_to
name
email
Contact_no
guest_id
Guest
address
1
M
1
booking_id
payment_status
Check_in_date
check_out_date
Booking
1
N
1
1
1
makes
writes
about
uses
places
N
N
N
N
service_charge
Service
service_name
service_id
feedback_id
Feedback
date
comments
rating
paid_by
1
N
order_date
Restaurant order
order_id
total_amount
1
paid_by
1
payment_id
payment
payment_date
amount
payment_mode

5. RELATIONAL SCHEMA



6. DDL COMMANDS (Sample)

```sql
CREATE TABLE Hotel (
    hotel_id INT AUTO_INCREMENT PRIMARY KEY,
    hotel_name VARCHAR(100) NOT NULL,
    location VARCHAR(100),
    rating DECIMAL(2,1)
);
```

```sql
INSERT INTO Hotel (hotel_name, location, rating) VALUES
('Taj Hotel', 'Mumbai', 4.8),
('Oberoi', 'Delhi', 4.6),
('Leela Palace', 'Bangalore', 4.7),
('Hyatt Regency', 'Chennai', 4.5),
('ITC Maratha', 'Pune', 4.4);
```

hotel_id	hotel_name	location	rating
1	Taj Hotel	Mumbai	4.8
2	Oberoi	Delhi	4.6
3	Leela Palace	Bangalore	4.7
4	Hyatt Regency	Chennai	4.5
5	ITC Maratha	Pune	4.4
NULL	NULL	NULL	NULL

```sql
CREATE TABLE Staff (
    staff_id INT AUTO_INCREMENT PRIMARY KEY,
    hotel_id INT,
    staff_name VARCHAR(100),
    role VARCHAR(50),
    phone VARCHAR(15),
    FOREIGN KEY (hotel_id) REFERENCES Hotel(hotel_id)
        ON DELETE CASCADE
);
```

```sql
INSERT INTO Staff (hotel_id, staff_name, role, phone) VALUES
(1, 'Amit Verma', 'Manager', '9876543210'),
(1, 'Sita Rao', 'Receptionist', '9876501234'),
(2, 'Raj Mehta', 'Chef', '9876512345'),
(3, 'Anjali Nair', 'Housekeeping', '9876523456'),
(4, 'Rohit Sharma', 'Receptionist', '9876534567');
```

room_id	hotel_id	room_type	price_per_night	status
1	1	Deluxe	5000.00	Available
2	1	Suite	8000.00	Booked
3	2	Standard	3000.00	Available
4	3	Luxury Suite	10000.00	Available
5	4	Executive	6000.00	Booked
NULL	NULL	NULL	NULL	NULL

```sql
CREATE TABLE Room (
    room_id INT AUTO_INCREMENT PRIMARY KEY,
    hotel_id INT,
    room_type VARCHAR(50),
    price_per_night DECIMAL(10,2),
    status ENUM('Available','Booked') DEFAULT 'Available',
    FOREIGN KEY (hotel_id) REFERENCES Hotel(hotel_id)
        ON DELETE CASCADE
);
```

```sql
INSERT INTO Room (hotel_id, room_type, price_per_night, status) VALUES
(1, 'Deluxe', 5000, 'Available'),
(1, 'Suite', 8000, 'Booked'),
(2, 'Standard', 3000, 'Available'),
(3, 'Luxury Suite', 10000, 'Available'),
(4, 'Executive', 6000, 'Booked');
```

guest_id	guest_name	phone	email	address
1	Rahul Sharma	9876543210	rahul@gmail.com	Delhi
2	Ananya Singh	9876501234	ananya@gmail.com	Mumbai
3	Arjun Mehta	9876512345	arjun@gmail.com	Bangalore
4	Sneha Kapoor	9456789123	sneha@gmail.com	Hyderabad
5	Vikram Jain	9876540987	vikram@gmail.com	Chennai
NULL	NULL	NULL	NULL	NULL

```sql
CREATE TABLE Guest (
    guest_id INT AUTO_INCREMENT PRIMARY KEY,
    guest_name VARCHAR(100),
    phone VARCHAR(15),
    email VARCHAR(100),
    address VARCHAR(200)
);
```

```sql
INSERT INTO Guest (guest_name, phone, email, address) VALUES
('Rahul Sharma', '9876543210', 'rahul@gmail.com', 'Delhi'),
('Ananya Singh', '9876501234', 'ananya@gmail.com', 'Mumbai'),
('Arjun Mehta', '9876512345', 'arjun@gmail.com', 'Bangalore'),
('Sneha Kapoor', '9456789123', 'sneha@gmail.com', 'Hyderabad'),
('Vikram Jain', '9876540987', 'vikram@gmail.com', 'Chennai');
```

booking_id	guest_id	room_id	check_in	check_out
1	1	1	2025-09-20	2025-09-22
2	2	2	2025-09-25	2025-09-28
3	3	3	2025-10-01	2025-10-03
4	4	4	2025-10-05	2025-10-07
5	5	5	2025-10-10	2025-10-12
NULL	NULL	NULL	NULL	NULL

```sql
CREATE TABLE Booking (
    booking_id INT AUTO_INCREMENT PRIMARY KEY,
    guest_id INT,
    room_id INT,
    check_in DATE,
    check_out DATE,
    FOREIGN KEY (guest_id) REFERENCES Guest(guest_id),
    FOREIGN KEY (room_id) REFERENCES Room(room_id)
);
```

```sql
INSERT INTO Booking (guest_id, room_id, check_in, check_out) VALUES
(1, 1, '2025-09-20', '2025-09-22'),
(2, 2, '2025-09-25', '2025-09-28'),
(3, 3, '2025-10-01', '2025-10-03'),
(4, 4, '2025-10-05', '2025-10-07'),
(5, 5, '2025-10-10', '2025-10-12');
```

booking_id	guest_id	room_id	check_in	check_out
1	1	1	2025-09-20	2025-09-22
2	2	2	2025-09-25	2025-09-28
3	3	3	2025-10-01	2025-10-03
4	4	4	2025-10-05	2025-10-07
5	5	5	2025-10-10	2025-10-12
NULL	NULL	NULL	NULL	NULL

```sql
CREATE TABLE Payment (
    payment_id INT AUTO_INCREMENT PRIMARY KEY,
    booking_id INT,
    amount DECIMAL(10,2),
    payment_date DATE,
    method ENUM('Cash','Card','UPI'),
    FOREIGN KEY (booking_id) REFERENCES Booking(booking_id)
);
```

```sql
INSERT INTO Payment (booking_id, amount, payment_date, method) VALUES
(1, 10000, '2025-09-19', 'Card'),
(2, 15000, '2025-09-24', 'UPI'),
(3, 8000, '2025-09-30', 'Cash'),
(4, 12000, '2025-10-04', 'Card'),
(5, 6000, '2025-10-09', 'UPI');
```

payment_id	booking_id	amount	payment_date	method
1	1	10000.00	2025-09-19	Card
2	2	15000.00	2025-09-24	UPI
3	3	8000.00	2025-09-30	Cash
4	4	12000.00	2025-10-04	Card
5	5	6000.00	2025-10-09	UPI
NULL	NULL	NULL	NULL	NULL

```sql
CREATE TABLE Service (
    service_id INT AUTO_INCREMENT PRIMARY KEY,
    service_name VARCHAR(100),
    price DECIMAL(10,2)
);
```

```sql
INSERT INTO Service (service_name, price) VALUES
('Spa', 2000),
('Laundry', 500),
('Airport Pickup', 1500),
('Room Service', 1000),
('Gym Access', 800);
```

service_id	service_name	price
1	Spa	2000.00
2	Laundry	500.00
3	Airport Pickup	1500.00
4	Room Service	1000.00
5	Gym Access	800.00
NULL	NULL	NULL

```sql
CREATE TABLE Guest_Service (
    guest_id INT,
    service_id INT,
    PRIMARY KEY (guest_id, service_id),
    FOREIGN KEY (guest_id) REFERENCES Guest(guest_id),
    FOREIGN KEY (service_id) REFERENCES Service(service_id)
);
```

```sql
INSERT INTO Guest_Service (guest_id, service_id) VALUES
(1, 1),
(1, 4),
(2, 2),
(3, 3),
(4, 1),
(5, 5);
```

guest_id	service_id
1	1
4	1
2	2
3	3
1	4
5	5
NULL	NULL

```sql
CREATE TABLE RestaurantOrder (
    order_id INT AUTO_INCREMENT PRIMARY KEY,
    guest_id INT,
    order_date DATE,
    total_amount DECIMAL(10,2),
    FOREIGN KEY (guest_id) REFERENCES Guest(guest_id)
);
```

```sql
INSERT INTO RestaurantOrder (guest_id, order_date, total_amount) VALUES
(1, '2025-09-21', 1200),
(2, '2025-09-26', 800),
(3, '2025-10-02', 1500),
(4, '2025-10-06', 1000),
(5, '2025-10-11', 1300);
```

order_id	guest_id	order_date	total_amount
1	1	2025-09-21	1200.00
2	2	2025-09-26	800.00
3	3	2025-10-02	1500.00
4	4	2025-10-06	1000.00
5	5	2025-10-11	1300.00
NULL	NULL	NULL	NULL

```sql
CREATE TABLE Feedback (
    feedback_id INT AUTO_INCREMENT PRIMARY KEY,
    booking_id INT,
    rating INT CHECK (rating BETWEEN 1 AND 5),
    comments TEXT,
    FOREIGN KEY (booking_id) REFERENCES Booking(booking_id)
);
```

```sql
INSERT INTO Feedback (booking_id, rating, comments) VALUES
(1, 5, 'Excellent service and very comfortable stay.'),
(2, 4, 'Good experience, room was clean.'),
(3, 3, 'Average stay, food could be better.'),
(4, 5, 'Loved the stay, staff was very helpful.'),
(5, 4, 'Nice hotel, but a bit noisy.');
```

feedback_id	booking_id	rating	comments
1	1	5	Excellent service and very comfortable stay.
2	2	4	Good experience, room was clean.
3	3	3	Average stay, food could be better.
4	4	5	Loved the stay, staff was very helpful.
5	5	4	Nice hotel, but a bit noisy.
NULL	NULL	NULL	NULL

7. TRIGGERS, PROCEDURES, AND FUNCTIONS

7.1 Triggers

after_booking_insert: Automatically updates room status to 'Booked' when new booking is created

before_payment_insert: Validates payment amount doesn't exceed booking total

after_payment_insert: Creates audit log entry for each payment transaction

update_hotel_rating: Recalculates hotel rating based on customer feedback

7.2 Stored Procedures

GetTotalPaymentByGuest(guestId): Calculate total payments made by a guest

GetAvailableRoomsByHotel(hotelId): Retrieve all available rooms for a hotel

AddBooking(guestId, roomId, checkIn, checkOut): Create booking with validation

CheckOutGuest(bookingId): Process guest checkout and release room

7.3 User-Defined Functions

StayDuration(bookingId): Calculate number of nights for a booking

TotalServiceCost(guestId): Sum all service charges for a guest

TotalBill(bookingId): Get total payment amount for booking

8. APPLICATION FEATURES AND FUNCTIONALITIES

1. **Home Dashboard:** Real-time metrics with total hotels, guests, bookings, and revenue
2. **Hotel Management:** Complete CRUD operations for hotels and rooms
3. **Guest Management:** Guest registration, service assignment, and history tracking
4. **Booking System:** Automated booking with room status updates and checkout
5. **Payment Processing:** Multi-method payments with validation and logging
6. **Staff Directory:** Employee management with role-based assignments
7. **Restaurant Orders:** Integrated meal order tracking and billing
8. **Service Management:** Amenity catalog and guest service assignments
9. **Feedback System:** Customer ratings with automatic hotel score updates
10. **Analytics Dashboard:** Interactive charts for revenue, occupancy, and performance

9. ADVANCED SQL QUERY EXAMPLES

9.1 Nested Query - Hotels Above Average Rating

```
SELECT hotel_name, rating FROM Hotel WHERE rating > (SELECT AVG(rating) FROM Hotel);
```

9.2 JOIN Query - Complete Booking Details

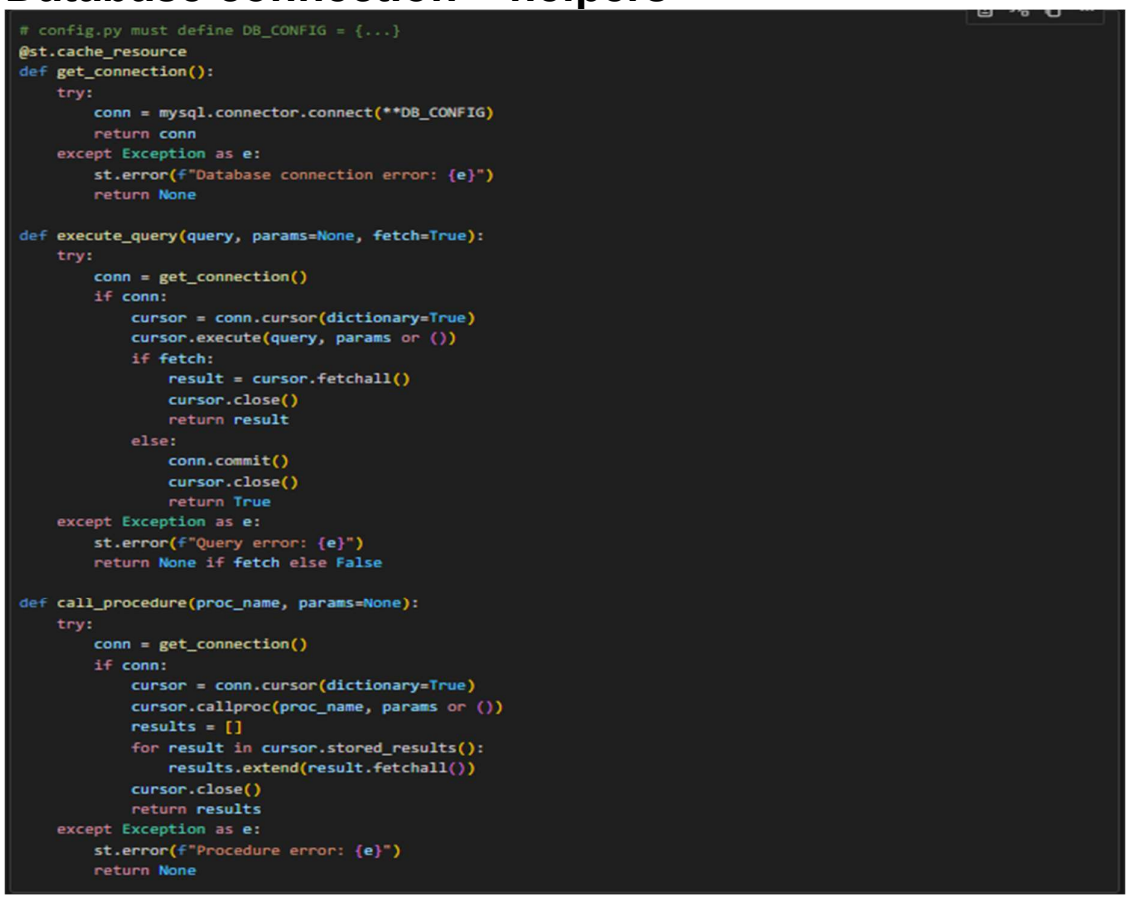
```
SELECT b.booking_id, g.guest_name, h.hotel_name, r.room_type, b.check_in,
b.check_out FROM Booking b JOIN Guest g ON b.guest_id = g.guest_id JOIN Room
r ON b.room_id = r.room_id JOIN Hotel h ON r.hotel_id = h.hotel_id;
```

9.3 Aggregate Query - Revenue by Hotel

```
SELECT h.hotel_name, COUNT(b.booking_id) as bookings, SUM(p.amount) as revenue FROM
Hotel h LEFT JOIN Room r ON h.hotel_id = r.hotel_id LEFT JOIN Booking b ON
r.room_id = b.room_id LEFT JOIN Payment p ON b.booking_id = p.booking_id GROUP BY
h.hotel_id, h.hotel_name ORDER BY revenue DESC;
```

10. Screen Shoot

1. Database connection + helpers

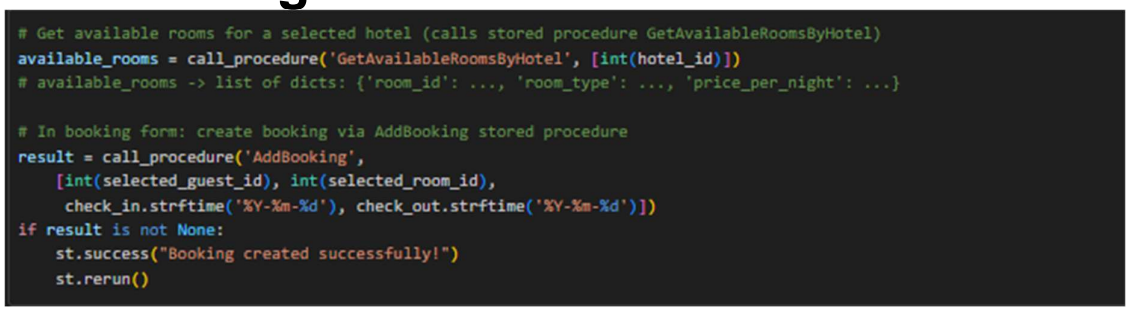


```python
# config.py must define DB_CONFIG = {...}
@st.cache_resource
def get_connection():
    try:
        conn = mysql.connector.connect(**DB_CONFIG)
        return conn
    except Exception as e:
        st.error(f"Database connection error: {e}")
        return None

def execute_query(query, params=None, fetch=True):
    try:
        conn = get_connection()
        if conn:
            cursor = conn.cursor(dictionary=True)
            cursor.execute(query, params or ())
            if fetch:
                result = cursor.fetchall()
                cursor.close()
                return result
            else:
                conn.commit()
                cursor.close()
                return True
    except Exception as e:
        st.error(f"Query error: {e}")
        return None if fetch else False

def call_procedure(proc_name, params=None):
    try:
        conn = get_connection()
        if conn:
            cursor = conn.cursor(dictionary=True)
            cursor.callproc(proc_name, params or ())
            results = []
            for result in cursor.stored_results():
                results.extend(result.fetchall())
            cursor.close()
            return results
    except Exception as e:
        st.error(f"Procedure error: {e}")
        return None
```

2. Booking



```python
# Get available rooms for a selected hotel (calls stored procedure GetAvailableRoomsByHotel)
available_rooms = call_procedure('GetAvailableRoomsByHotel', [int(hotel_id)])
# available_rooms -> list of dicts: {'room_id': ..., 'room_type': ..., 'price_per_night': ...}

# In booking form: create booking via AddBooking stored procedure
result = call_procedure('AddBooking',
    [int(selected_guest_id), int(selected_room_id),
     check_in.strftime('%Y-%m-%d'), check_out.strftime('%Y-%m-%d')])
if result is not None:
    st.success("Booking created successfully!")
    st.rerun()
```

3. Check-out



```python
# Check out a guest and free the room
call_procedure('CheckOutGuest', [int(booking_id)])
st.success("Guest checked out successfully! Room is now available.")
st.rerun()
```

4. Payment

```python
# Prepare suggested_amount from a SELECT (example shown earlier)
# Process payment (app-level validation added)
query = "INSERT INTO Payment (booking_id, amount, payment_date, method) VALUES (%s, %s, %s, %s)"
try:
    if execute_query(query, (int(booking_id), float(amount), payment_date, method), fetch=False):
        st.success(f"Payment of ₹{amount:,.2f} processed successfully!")
        st.balloons()
        st.rerun()
except Exception as e:
    st.error(f"Payment failed: {e}")
```

5. Guest page

```python
# Procedure call (function) call_procedure: Any
total_payment = call_procedure('GetTotalPaymentByGuest', [int(guest_id)])
if total_payment:
    st.metric("Total Payments", f"₹{total_payment[0].get('total_payment', 0):,.2f}")

# SQL function call via execute_query (TotalServiceCost is a DB function)
service_cost = execute_query("SELECT TotalServiceCost(%s) as cost", (int(guest_id),))
if service_cost:
    st.metric("Total Service Cost", f"₹{service_cost[0]['cost']:,.2f}")
```

6. Feedback submission

```python
# Insert feedback — this INSERT will fire update_hotel_rating trigger in DB
query = "INSERT INTO Feedback (booking_id, rating, comments) VALUES (%s, %s, %s)"
if execute_query(query, (int(booking_id), int(rating), comments), fetch=False):
    st.success("Feedback submitted successfully! Hotel rating will be updated automatically.")
    st.rerun()
```

11. GITHUB REPOSITORY

Repository URL: https: //github.com/akshayag2005/HOTEL-MANAGEMENT-SYSTEM

Repository Contents:
• app.py - Main Streamlit application
• config.py - Database configuration
• Hotel_management2.sql - Complete SQL scripts
• requirements.txt - Python dependencies
• LAB_REPORT.md - Documentation

12. CONCLUSION

The Hotel Management System successfully demonstrates practical application of database management concepts including relational design, normalization, triggers, stored procedures, and complex SQL queries. The integration with Streamlit provides an intuitive interface for all CRUD operations and analytics.

Key Achievements:

- Implemented normalized database schema (3NF)
- Created 10 interconnected tables with proper relationships
- Developed 4 triggers, 4 stored procedures, 3 functions
- Implemented comprehensive CRUD operations
- Designed responsive web interface with analytics
- Maintained data integrity through constraints

Future Enhancements: Online booking portal, email notifications, payment gateway integration, mobile application, multi-language support.